



Vaya – Engineering Home Assignment

This assignment, including its contents, questions, and any accompanying materials, is proprietary and confidential to Vaya and is provided solely for the purpose of evaluating the candidate's qualifications; it may not be copied, distributed, or shared with any third party. By accepting and submitting this assignment, the candidate acknowledges and agrees that any work product, analysis, code, designs, or other materials created in connection with it are the sole and exclusive property of Vaya, and the candidate hereby waives and releases any claim to intellectual property rights, compensation, or ownership interest therein

ABOUT VAYA

A car subscription service. You pick a car, you pay one monthly price with insurance and maintenance in it, you can hand it back. No dealership, no six-year loan. Dealers supply cars from stock that's otherwise sitting on the lot. We're running a pilot with one dealer and fifteen cars. A car is a physical object: it exists once, it's in one place, and it can't be in two agreements at the same time. And at the end of every month we take real money off a real card and sometimes have to defend the number in an email.

Use AI. I mean it.

Claude Code, Cursor, Codex, whatever you're fastest in. It's how I work, it's how you'd work here, and testing you without it would tell me nothing I want to know. You get no credit for suffering. Two things back in exchange.

HOW-I-BUILT-IT document

- **How you set the model up.** Rules files, custom instructions, subagents, MCP servers, whatever your setup is. Paste the files if you have them.
- **Where it was wrong.** At least one specific instance. An API it invented, a schema it got backwards, an approach it argued for that you rejected. What the wrong output was and how you noticed.
- **What you threw away.** Roughly how much, and why.
- **What you checked by hand,** and why that thing and not something else. This is the one I care about most. Deciding what needs human verification is the actual skill.
- **What you'd have done differently** with the tooling, knowing what you know at the end.



A video

Ten minutes, one take, no editing, no script. Share your screen and talk. Loom, QuickTime, anything.

1. Show it running. Click through both parts.
2. Pick the decision you were least sure about and talk me through the version you didn't go with.
3. Show me something the AI got wrong, and what you did about it.

I'm not judging the presentation and I don't care if you say "um". Ten minutes of you talking about your own work tells me more than another thousand lines would.

Stack

Yours. We're Next.js, TypeScript, Postgres, Prisma, so that's the path of least explanation, but pick what you're fast on. I care about the reasoning, not the framework.

Part 1 – build the marketplace

Two sides, one physical constraint between them.

Architecture – THE MOST IMPORTANT PART

I want to see how it's put together and why, in a **DECISIONS** document, first section. A diagram if it helps you, prose if it doesn't.

- The pieces, and what talks to what. The architecture must be much wider than the actual build in the next section.
- Where the data model came out different from the obvious shape, and what forced it.
- **Three or four real forks, with the road not taken.** What you picked, what you rejected, and what the rejected one would have cost. Not "I chose Postgres because it's reliable", the ones where you genuinely hesitated.
- Where this design breaks if the fleet goes from 15 cars to 5,000, and which part you'd replace first.

That last one is not an invitation to build for 5,000. Build for 15. Tell me where it snaps.



1 · Supply and demand

A driver sees what's actually available and commits to a car on a monthly plan. An ops-side view shows the state of the fleet.

2 · The invariant

A car carries at most one live commitment. Ever. Under any sequence of events.

If two commitments for the same car are posted simultaneously, exactly one must win, and the loser must get a sensible answer rather than a 500.

Include whatever you'd use to convince yourself this holds, and demonstrate it in the video.

3 · One mid-flight change

Live subscriptions get mutated. Pick **one** and do it properly:

- swap the driver to a different car, or
- change the plan tier mid-cycle, or
- end the subscription early.

You don't have to move money. You do have to know what's owed and be able to **show your work**, so that when someone emails asking why they were charged that, a person can open the record and answer them.

The data

`seed.json` ships with this. Load it as it is.

Don't clean it by hand. It contradicts itself in about ten places, the way production data does. Some have an obvious right answer, some have three defensible ones, and at least one breaks the rule you've just been asked to enforce. Your system decides what to do with each, and your ops screen says what it decided.

I'm not looking for you to fix the data. I'm looking at what you do when the data is wrong and the page still has to render. Refusing, quarantining, flagging for a human, picking a winner and saying so out loud, all defensible. Silently averaging over it isn't.



Part 2 – telemetry

Product sense focus, and some code. The memo is worth more than the build, and the build is still real.

The situation

The situation Fifteen cars, in the hands of subscribers, for months at a time. We know almost nothing about them once they leave the lot. Three things this costs us right now.

Billing. Our plans include a mileage allowance and we bill for the overage. The example our billing is pinned against: 6,840 miles driven, 840 over, \$252.00 charged, taken off a saved card. That mileage comes from a person reading the odometer at handover and reading it again at return, writing both numbers down. It works. It is also two manual readings months apart, and when a customer emails to say the charge is wrong, what we have is a number somebody typed.

Insurance. After the cars themselves, this is our largest cost. The fleet insurer prices our entire book at a single rate, because we cannot tell them one useful thing about how any individual drives. Our careful subscribers pay for our careless ones, we cannot identify either group, and we have no way to argue the rate down at renewal.

Things we cannot answer. A car came back with damage and nobody could say when it happened or who was driving. Another sat at an address ninety miles outside its plan area for three weeks and we found out when it came back. A subscriber disputed an overage charge and the only evidence we had was our own handwriting.

The memo

One to two pages.

1. Map what is actually possible. There is more than one way to get data out of a car and they are not variations on each other. Find the routes, and be concrete about what



each one physically can and cannot deliver. I am as interested in what is unavailable at any price as in what is on the menu.

2. Cost it, for fifteen cars. Per car, per month, first year, including anything that is not on the vendor's pricing page. Say which numbers you confirmed and which you are guessing.
3. Tell me what is worth paying for. The cheap end and the expensive end of this market differ by several times per car. Somewhere in that gap are features worth the money and features that are not, and I cannot tell which from a pricing page. This is the part I will read most carefully, because it is the part I cannot do myself.
4. Decide. One recommendation, not three options and a summary. Tie it to the three costs above and tell me which one you are actually attacking, and which you are choosing to leave alone for now. If your recommendation changes what anybody pays, say what makes that lawful where we operate. If your answer is that we should not do this yet, say that. I will take it seriously.
5. What you would need to prove before spending money, and how you would close it inside a day. Cheapest thing that settles the biggest question.
6. Name one thing you would refuse to build. There is more than one candidate.

Every fact should be traceable to something you actually read. Tell me where the numbers came from and mark the ones you could not confirm.

The build

Not the whole system. These four things, and nothing you do not need.

feed.jsonl ships with this. It is a month of output from one vendor we were pointed at, in their documented webhook shapes, plus a few records pulled from their REST API. Read it before you write the memo, not after. It is evidence, and some of what you need for question 1 is visible in the fields that are there and in the fields that are not.

The schema. Real, migrated, running. This is the centre of the exercise. The data arrives in shapes that punish the obvious model, and the obvious model corrupts billing quietly rather than loudly.



The ingestion. Whatever consumes that feed and turns it into a number you would put on an invoice. Work out from the file itself what it does to you. Some of it is documented vendor behaviour, some of it is what happens when a device spends two days out of signal, and one of it is what happens when somebody moves a device from one car to another without telling you.

One screen. The one an operator opens at nine in the morning when a customer emails saying the charge is wrong. Not a dashboard. Show it being used for that actual task in the video.

Tests that prove the dirty data is handled. One per failure mode you found. I want to see the ingestion fed something malformed and behaving.

Then tell me, in the memo, what you deliberately left out and why. You are choosing the scope inside those four items and the choice is part of what I am reading.

The four rules

These are the rules we actually work under, both parts.

One command to set up, one to run. I'll run them on a clean machine and click around. If that doesn't work, that's the result. Say which OS you built on.

No empty screens. Every screen you build has rows behind it when I open it. This is the single most common way things fail here: a page that works perfectly with nothing in it. I've stopped treating it as a data problem and started treating it as part of building the screen.

No numbers you can't trace. Every figure on screen comes from a row. If you can't compute it honestly, leave it out and say why. A plausible number nobody can defend is worse than a blank.

Write down what you decided. **DECISIONS:** architecture first (above), then the calls you made where I didn't specify, what you rejected and what it would have cost, what you'd have asked me if we were on Slack, and what you'd do next with another day, in order.



Deliver

A git repo, private link or zip:

the code	both parts
README.md	the two commands, and which OS you built on
DECISIONS	architecture, then decisions
HOW-I-BUILT-IT	as specified above
the memo	markdown or PDF
the video	a link

How I'll read it

The memo	Did you decide. Did you find vendors we didn't. Can you tell me what's worth paying for.
The architecture	The forks, and whether the rejected options had real appeal.
Does the invariant hold	I'll attack it. Where you enforced it matters more than that you did.
The schema in Part 2	Whether it survives a device moving between cars.
What you did with the bad data	Both parts. Refuse, quarantine, flag, guess — any can be right. Smoothing it over isn't.



Does it open full	I run it and click.
HOW-I-BUILT-IT	Can you show me a moment where you overruled the model.
The video	Ten minutes.

Ask me anything. It costs you nothing and I'd rather answer than have you guess.